

Ficha de atividade 2 – Criptografia e Segurança em Redes (2024/25)

Nome: Luís Pedro Cerqueira Baptista – A105419

Data: 25/09/2024

Exercícios:

Exercício 1 -> (Criptanálise da Cifra de César):

Mostre, com auxílio de funções Python , como quebrar o criptograma MESUAXJSKAEHDWKVWIMWTJSJ .

Qual foi a chave utilizada para produzir este criptograma?

Exercício 2 -> Crie um programa em python que implemente a Cifra de Vigenère (apresentada na aula teórica).

O programa deverá receber três parâmetros:

1. corresponde ao texto (original ou criptograma),
2. corresponde a chave
3. operação a executar (cifrar ou decifrar).

```
$ python Vigenere.py "texto a cifrar" SEGREDO cifrar
```

```
LIDKSDQAJXRV
```

```
$ python Vigere.py LIDKSDQAJXRV SEGREDO decifrar
```

```
TEXTTOACIFRAR
```

Exercício 3 -> Modifique as implementações Client.py e Server.py para suportar comunicação segura usando a cifra Fernet (ver documentação neste link-[Fernet \(symmetric encryption\) — Cryptography 44.0.0.dev1 documentation](#))

➔ Como contornar a necessidade de partilha prévia da chave?

Resolução:

Exercício 1 - > Quebrar o criptograma MESUAXJSKAEHDWKVWIMWTJSJ

1. Criar uma função em C ou Python
2. Testar diferentes shifts
3. Obter um texto legível

Estrutura do código:

- Definir o criptograma a ser decifrado/quebrado.
(MESUAXJSKAEHDWKVWIMWTJSJ)
- Ciclo for para testar todas as chaves (0 a 25)

→ Função decifrar:

- Para cada caractere do texto:

1 - Calcular o novo valor do caractere usando o Shift inverso.

```
char_decifrado = chr((ord(caractere_cifrado) - 65 - chave) % 26 + 65)
```

→ Fim do Programa:

Chamar a função decifrar

Ficheiro com a resolução: Exercicio_1

Exercício 2 - > Cifragem de Vigenère

1. Criar programa em python com os parâmetros necessários para cumprir os requisitos da cifra de Vigenère
- Criptograma
 - Chave Secreta
 - Cifragem/Decifragem

Estrutura do Código:

Parâmetros – função Vigenère – ciclo “for” para operações Shift – usar as operações (cifrar/decifrar)

Ficheiro com a resolução: Exercicio_2

Exercício 3 - Cifragem Fernet

1. Modificar os scripts Client.py e Server.py para suportar comunicação segura com a cifra Fernet
2. Usar uma chave hardcoded (32bytes em Base64 (ASCII) codificada e válida para o Fernet) onde o cliente encripta e envia a mensagem (encriptada) para o servidor onde é desencriptada com a mesma chave do cliente
3. Executar o Servidor e depois o cliente para obter a mensagem!

Riscos associados: O uso de uma chave hardcoded retira a necessidade de uma troca prévia de chave entre cliente – servidor, mas acaba por comprometer a segurança da secret key!

Estrutura do Código: Explicada nos ficheiros!

Ficheiro com a resolução: Exercicio_3.1 (cliente.py) e Exercicio_3.2 (server.py)